

Expressions, Types, and Variables

CSCI 1101

2024-01-25

Expressions and Values

An **expression** represents information in a form that a computer can understand.

For example, a calculator understands **numeric expressions**.

$$2 + 2$$

$$5 - 1$$

The simplified form of an expression is called a **value**.

$$2 + 2 = 4$$

Often the *same value* can be represented using *different expressions*.

$$2 + 2 = 5 - 1$$

Like a calculator, Python knows how to **evaluate** numeric expressions to their values.

Activity: Evaluating Expressions

Use the **Python interactive interpreter** on your computer (IDLE or Wing101) to evaluate the following expressions:

```
2 + 2      # addition
5 - 1      # subtraction
3 * 4      # multiplication
2 ** 3     # exponentiation
10 // 3    # quotient
10 % 3     # remainder
```

Text on a line after a “#” is a **comment**, which is ignored by Python.

Types

Python can understand many kinds of information besides numbers.

Every Python expression has a **type**, which classifies what kind of information it represents.

To find out the type of an expression, you can ask the interpreter using:

```
type(<expression>)
```

For example:

```
type(2 + 2)
```

Rule: the type of an expression is always the same as the type of its value.

Numeric Types

In Python the type of integers (whole numbers) is called “`int`”.

Python also has another numeric type.

Ask Python to evaluate the following expression: `10 / 3`

Do you notice anything strange?

Python has a type of *approximate* decimal numbers called “`float`”.

Internally they are stored using (binary) *scientific notation*, with a fixed budget for both the precision (significant digits) and the magnitude (exponent).

Python distinguishes `floats` from `ints` using a decimal point (e.g. “`4.0`” vs “`4`”).

Overloading Notation

For the convenience of humans, Python lets us **overload** notation.

This means using the *same syntax* to mean to *different things* in different contexts.

Ask Python to evaluate the following expressions:

`2.5 + 2.5`

`5.0 - 1.0`

`3.0 * 4.0`

You should be able to guess their types by the way their values are displayed.

Compound Expressions

We can build up larger expressions from smaller ones by grouping them with parentheses.

$$2 * (3 + 4)$$

For convenience we may drop some parentheses according to the standard conventions for *precedence* and *associativity* (see homework).

What are the values of the following expressions?

$$2 * 3 + 4$$

$$2 + 2 + 2$$

When in doubt, you can always put in the implicit parentheses explicitly to make the structure of an expression unambiguous.

Variable Assignment

Python can not only evaluate an expression, but also remember the value for later use.

We associate the value of an expression with a **variable** using an **assignment statement**.

This is written according to the following pattern:

```
<name_of_variable> = <expression>
```

For example:

```
life_the_universe_and_everything = 6 * 7
```

There are a few rules and conventions for picking variable names (see homework).

You should pick variable names that are mnemonic for the *meaning* of their value.

Variable Assignment Algorithm

When Python encounters a variable assignment statement the following steps take place:

1. The expression on the right of the “=” is evaluated to a value.
2. That value becomes the value of the variable on the left of the “=”.

Activity: Variable Assignment

Using the variable assignment algorithm, predict the values of the following variables after each assignment statement:

```
x = 2 * 3  
y = x  
y = y + 1  
z = x * y
```

Note that the “=” of variable assignment is *not* the mathematical notion of “equals” (for example, the equation $y = y + 1$ has no integer solutions).

I encourage you to read variable assignment as “gets” to reinforce this distinction.

To Do:

- Ensure that Python (and optionally Wing 101) are installed on your computer,
- If you have not already done so, join the Slack workspace,
- Sign into CodeRunner (if it asks for a code, use “2024SP1101”) and make sure to complete the following tasks by Monday:
 - *Homework on Collaboration Policy*
 - *Homework 0: Intro to CodeRunner*
- Begin working on *Homework 1: Expressions, Variables, and Functions* in CodeRunner (we will learn about functions next time).